

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Tecnología en Análisis y Desarrollo de Software

INFORME TÉCNICO

Actividad:

Producto

Proyecto:

NexCode – Sistema de Gestión de Proyectos de Software

Autor:

Ivan Dario Florez Hernandez

Instructor:

Richard Alexander Betancur Sierra

Fecha:

11/06/2026

1. Introducción

NexCode permite gestionar de forma integral los elementos fundamentales del ciclo de vida de un proyecto de software: proyectos, desarrolladores, sprints y bugs. Cada uno de estos módulos fue implementado como un conjunto de operaciones CRUD (Create, Read, Update, Delete) expuestas a través de endpoints HTTP, siguiendo las convenciones de la arquitectura REST.

La solución prescinde de un motor de base de datos relacional o documental, optando en su lugar por la persistencia de datos mediante archivos JSON. Esta decisión arquitectónica, si bien orientada al contexto académico, permite comprender los mecanismos de lectura y escritura de datos estructurados en el sistema de archivos del servidor, así como la serialización y deserialización de objetos JavaScript.

2. Objetivos

2.1 Objetivo General

Desarrollar una API REST funcional denominada NexCode, utilizando Node.js y Express.js, que permita la gestión integral de proyectos de software, incluyendo el control de proyectos, desarrolladores, sprints y bugs, implementando reglas de negocio definidas, persistencia en archivos JSON y documentación Swagger.

2.2 Objetivos Específicos

- Implementar los módulos CRUD para las entidades Proyecto, Desarrollador, Sprint y Bug, siguiendo los principios de la arquitectura REST.
- Definir y aplicar reglas de negocio que garanticen la integridad referencial entre las entidades del sistema.
- Configurar la persistencia de datos mediante archivos JSON estructurados para cada entidad.
- Documentar todos los endpoints de la API utilizando Swagger/OpenAPI, garantizando la trazabilidad de las operaciones disponibles.
- Validar el funcionamiento de los endpoints mediante pruebas funcionales ejecutadas con Thunder Client.
- Aplicar buenas prácticas de estructuración de proyectos Node.js, incluyendo la separación de responsabilidades mediante el uso de rutas y controladores modulares.

3. Análisis de Requerimientos

3.1 Requerimientos Funcionales

RF #	Descripción	Módulo
RF 01	El sistema debe permitir crear, consultar, actualizar y eliminar proyectos.	Proyectos
RF 02	El sistema debe permitir crear, consultar, actualizar y eliminar desarrolladores.	Desarrolladores
RF 03	El sistema debe permitir crear, consultar, actualizar y eliminar sprints.	Sprints
RF 04	El sistema debe permitir crear, consultar, actualizar y eliminar bugs.	Bugs
RF 05	No se permite eliminar un proyecto que tenga sprints activos.	Proyectos / Sprints
RF 06	No se permite crear sprints para proyectos en estado 'entregado'.	Sprints
RF 07	No se permite crear un sprint para un proyecto inexistente.	Sprints
RF 08	No se permite registrar bugs para proyectos inexistentes.	Bugs
RF 09	No se permite asignar bugs a desarrolladores inexistentes.	Bugs

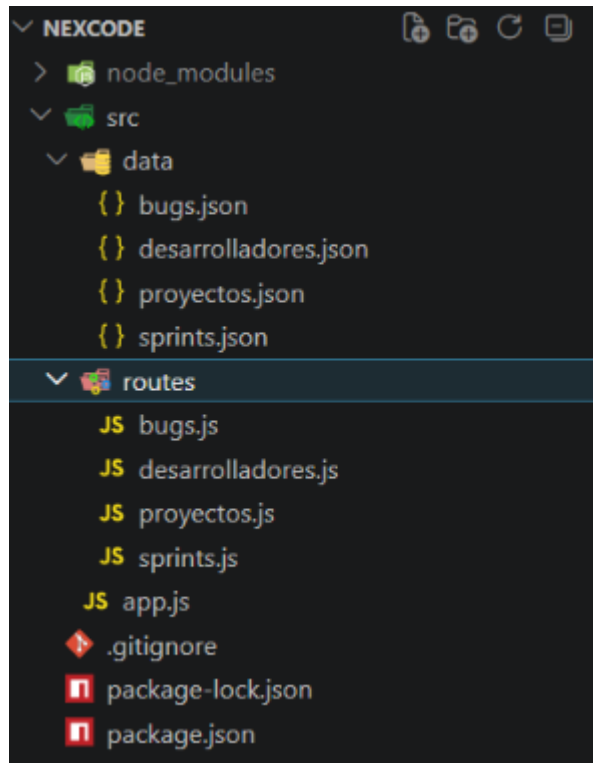
3.2 Requerimientos No Funcionales

RN F#	Descripción	Categoría
RN F01	La API debe responder en formato JSON para todas sus operaciones.	Interoperabilidad
RN F02	Los datos deben persistir en archivos JSON almacenados localmente.	Persistencia
RN F03	El proyecto debe estructurarse de manera modular con separación de rutas.	Mantenibilidad
RN F04	Los errores deben retornar códigos HTTP apropiados (400, 404, 500).	Robustez
RN F05	La documentación Swagger debe mantenerse actualizada con todos los endpoints.	Documentación

4. Arquitectura de la Solución

NexCode implementa una arquitectura REST (Representational State Transfer) sobre un servidor HTTP levantado con Express.js. La estructura del proyecto sigue el patrón de separación por responsabilidades, organizando el código en capas claramente definidas:

Estructura del Proyecto:

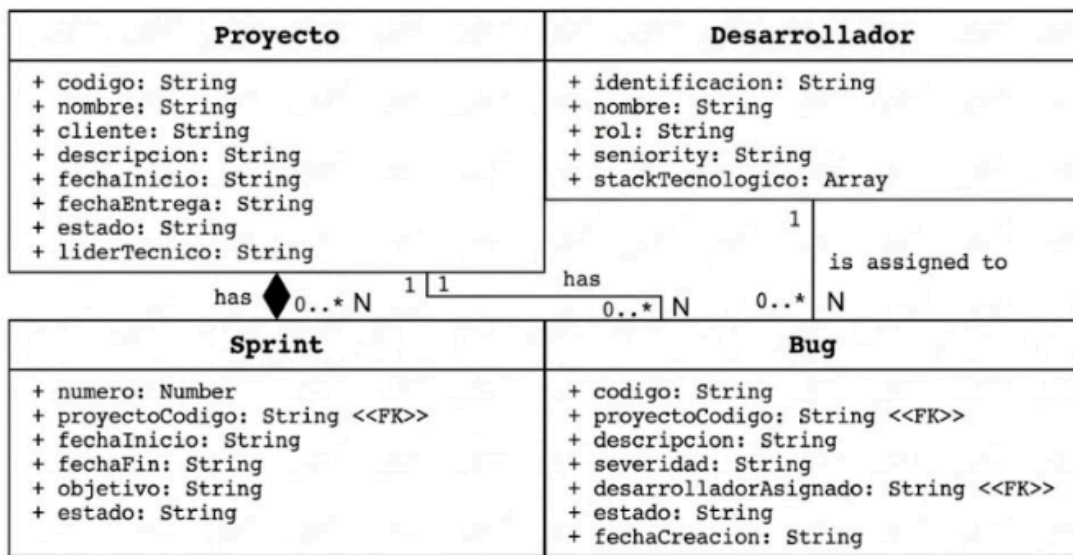


4.1 Flujo de una Petición HTTP

El flujo de procesamiento de una petición HTTP en NexCode sigue la siguiente secuencia: el cliente (Thunder Client, navegador u otra aplicación) envía una petición HTTP al servidor Express.js. El servidor evalúa la ruta y el método HTTP para despachar la petición al router correspondiente. El router ejecuta la lógica de negocio, que incluye la validación de reglas, la lectura o escritura en el archivo JSON correspondiente y la construcción de la respuesta. Finalmente, Express.js serializa la respuesta en formato JSON y la retorna al cliente con el código de estado HTTP apropiado.

5. Diagrama de Clases

El siguiente diagrama representa la estructura de clases del sistema NexCode, incluyendo los atributos de cada entidad y las relaciones de asociación entre ellas:



Relaciones entre entidades: Un Proyecto puede tener múltiples Sprints (relación 1 a N); Sprint referencia a Proyecto mediante el atributo proyectoCodigo. Un Proyecto puede tener múltiples Bugs (relación 1 a N); Bug referencia a Proyecto mediante proyectoCodigo. Un Desarrollador puede tener múltiples Bugs asignados (relación 1 a N); Bug referencia a Desarrollador mediante desarrolladorAsignado.

6. Implementación

6.1 CRUD de Proyectos

El módulo de Proyectos expone cuatro endpoints principales sobre la ruta base /api/proyectos. El endpoint POST /api/proyectos recibe el cuerpo de la petición con los datos del nuevo proyecto, valida los campos obligatorios y persiste el registro en el archivo proyectos.json. GET /api/proyectos retorna la lista completa de proyectos almacenados. GET /api/proyectos/:codigo retorna un proyecto específico identificado por su código. PUT /api/proyectos/:codigo actualiza los campos del proyecto indicado. DELETE /api/proyectos/:codigo elimina el proyecto, previa validación de que no existan sprints activos asociados.

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/proyectos`. The response status is 200 OK, with a size of 397 Bytes and a time of 116 ms. The response body is a JSON array containing two project objects.

```
1 [
2   {
3     "codigo": 1,
4     "nombre": "Sistema ERP",
5     "cliente": "Empresa ABC",
6     "descripcion": "ERP empresarial",
7     "fechaInicio": "2026-06-10",
8     "fechaEntrega": "2026-12-10",
9     "estado": "planificacion",
10    "liderTecnico": "Carlos Perez"
11  },
12  {
13    "codigo": 2,
14    "nombre": "Alerta Urbana",
15    "cliente": "Yo",
16    "descripcion": "Proyecto Final ",
17    "fechaInicio": "2026-06-10",
18    "fechaEntrega": "2026-12-10",
19    "estado": "En desarrollo",
20    "liderTecnico": "Ivan Florez"
21  }
22 ]
```

6.2 CRUD de Desarrolladores

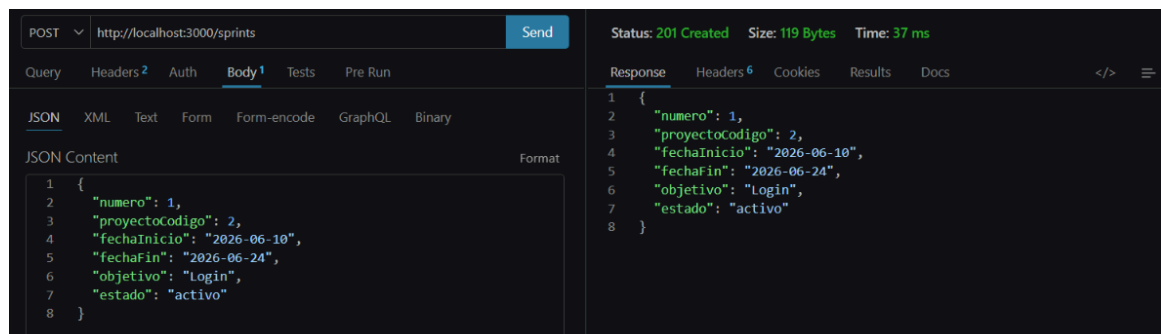
El módulo de Desarrolladores opera sobre la ruta base /api/desarrolladores. El endpoint POST /api/desarrolladores registra un nuevo desarrollador con su información de identificación, rol, nivel de seniority y stack tecnológico. Los endpoints GET permiten recuperar la lista completa o un desarrollador específico por su número de identificación. PUT actualiza la información del desarrollador y DELETE lo elimina del sistema. Antes de cualquier asignación de bug, el sistema verifica la existencia del desarrollador en este archivo.

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/desarrolladores`. The response status is 201 Created, with a size of 163 Bytes and a time of 1.03 s. The response body is a JSON object containing a message and a developer object.

```
1 {
2   "mensaje": "Desarrollador creado",
3   "desarrollador": {
4     "identificacion": 1001,
5     "nombre": "Ivan Florez",
6     "rol": "backend",
7     "seniority": "Junior",
8     "stackTecnologico": "Node.js"
9   }
10 }
```

6.3 CRUD de Sprints

El módulo de Sprints opera sobre `/api/sprints`. Al crear un sprint mediante POST, el sistema valida dos reglas de negocio críticas: que el proyecto referenciado exista en `proyectos.json` y que su estado no sea 'entregado'. Si alguna de estas condiciones no se cumple, el endpoint retorna un error HTTP 400 con un mensaje descriptivo. Los sprints se recuperan, actualizan y eliminan mediante sus respectivos endpoints, y su estado puede evolucionar de 'activo' a 'completado' o 'cancelado'.

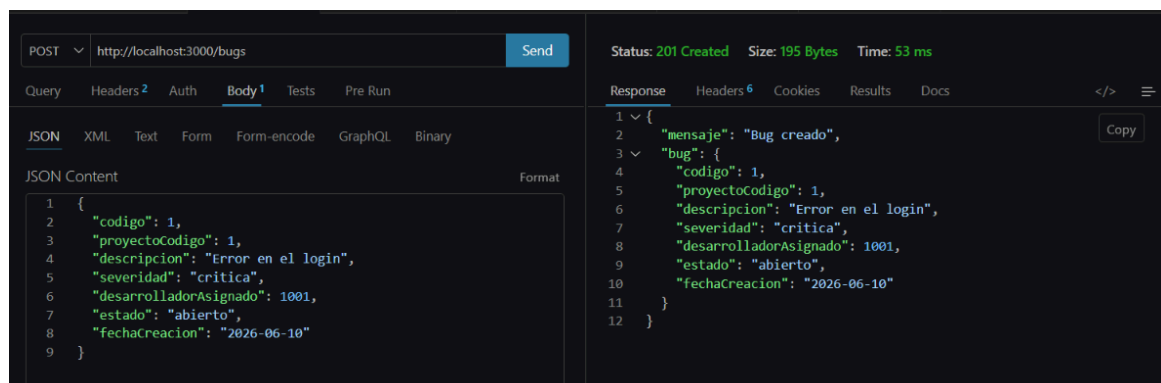


```
POST http://localhost:3000/sprints
{
  "numero": 1,
  "proyectoCodigo": 2,
  "fechaInicio": "2026-06-10",
  "fechaFin": "2026-06-24",
  "objetivo": "Login",
  "estado": "activo"
}
```

```
201 Created
{
  "numero": 1,
  "proyectoCodigo": 2,
  "fechaInicio": "2026-06-10",
  "fechaFin": "2026-06-24",
  "objetivo": "Login",
  "estado": "activo"
}
```

6.4 CRUD de Bugs

El módulo de Bugs opera sobre `/api/bugs`. El registro de un nuevo bug mediante POST requiere que el `proyectoCodigo` referenciado exista. Si se proporciona un `desarrolladorAsignado`, el sistema verifica adicionalmente que ese identificador corresponda a un desarrollador registrado en el sistema. Los bugs contienen información de severidad (baja, media, alta, critica) y su estado puede gestionarse a través del tiempo desde 'abierto' hasta 'resuelto' o 'cerrado'.



```
POST http://localhost:3000/bugs
{
  "codigo": 1,
  "proyectoCodigo": 1,
  "descripcion": "Error en el login",
  "severidad": "critica",
  "desarrolladorAsignado": 1001,
  "estado": "abierto",
  "fechaCreacion": "2026-06-10"
}
```

```
201 Created
{
  "mensaje": "Bug creado",
  "bug": {
    "codigo": 1,
    "proyectoCodigo": 1,
    "descripcion": "Error en el login",
    "severidad": "critica",
    "desarrolladorAsignado": 1001,
    "estado": "abierto",
    "fechaCreacion": "2026-06-10"
  }
}
```

7. Pruebas Funcionales

Las pruebas funcionales de NexCode fueron ejecutadas mediante Thunder Client, cliente HTTP integrado en Visual Studio Code. Se diseñaron dos pruebas funcionales completas que cubren los flujos principales de la aplicación, incluyendo la verificación de las reglas de negocio implementadas.

7.1 Prueba Funcional 1: Flujo Completo de Gestión de Proyecto con Bug Crítico

Campo	Detalle
Objetivo	Verificar el flujo completo de creación de un proyecto, sus desarrolladores, sprints y el ciclo de vida de un bug crítico desde su registro hasta su resolución.
Datos de Entrada	Proyecto: código 3, nombre 'NexCode MVP', cliente 'TechCorp SAS', estado 'activo'. Desarrollador: identificación 1002, nombre 'Pedro agudelo', rol 'backend', seniority 'senior'. Sprint 1: proyectoCodigo 1, estado 'activo'. Bug: código 2, severidad 'critica', proyectoCodigo 3, desarrolladorAsignado 1002.
Procedimiento	1. POST /api/proyectos – Crear proyecto PRY-001. 2. POST /api/desarrolladores – Crear desarrolladora Ana Gómez. 3. POST /api/sprints – Crear Sprint 1. 4. POST /api/sprints – Crear Sprint 2. 5. POST /api/bugs – Registrar bug crítico BUG-001 sin desarrollador asignado. 6. PUT /api/bugs/BUG-001 – Asignar bug al desarrollador 1001. 7. PUT /api/bugs/BUG-001 – Cambiar estado a 'resuelto'.
Resultado Esperado	Todos los recursos deben crearse con respuesta HTTP 201. La asignación del bug debe retornar HTTP 200. El cambio de estado a 'resuelto' debe retornar HTTP 200. En ningún paso debe presentarse un error 400 o 404.
Resultado Obtenido	Todos los endpoints respondieron con los códigos HTTP esperados. Los archivos JSON fueron actualizados correctamente en cada operación. El bug BUG-001 refleja estado 'resuelto' y desarrolladorAsignado '1001' en bugs.json.

POST

http://localhost:3000/proyectos

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```
1 {
2   "codigo": 1,
3   "nombre": "NexCode MVP",
4   "cliente": "TechCorp SAS",
5   "descripcion": "ERP empresarial",
6   "fechaInicio": "2026-06-10",
7   "fechaEntrega": "2026-12-10",
8   "estado": "Activo",
9   "liderTecnico": "Ivan Florez"
10 }
```

Status: 201 Created

Size: 235 Bytes

Time: 89 ms

Response

Headers 6

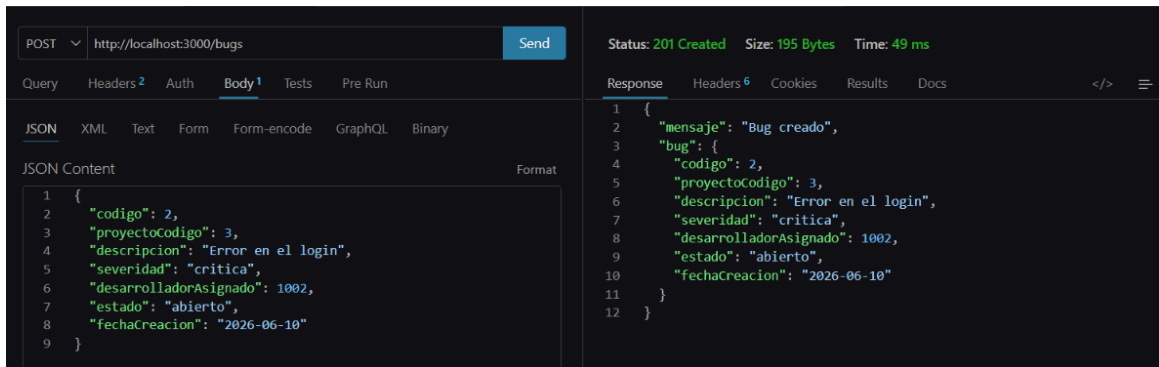
Cookies

Results

Docs

</>

```
1 {
2   "mensaje": "Proyecto creado",
3   "proyecto": {
4     "codigo": 1,
5     "nombre": "NexCode MVP",
6     "cliente": "TechCorp SAS",
7     "descripcion": "ERP empresarial",
8     "fechaInicio": "2026-06-10",
9     "fechaEntrega": "2026-12-10",
10    "estado": "Activo",
11    "liderTecnico": "Ivan Florez"
12  }
13 }
```

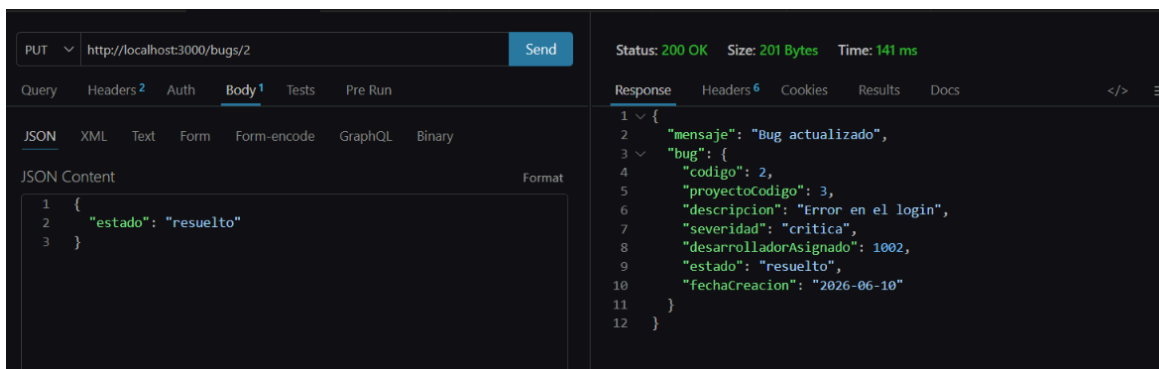



```

POST http://localhost:3000/bugs
{
  "codigo": 2,
  "proyectoCodigo": 3,
  "descripcion": "Error en el login",
  "severidad": "critica",
  "desarrolladorAsignado": 1002,
  "estado": "abierto",
  "fechaCreacion": "2026-06-10"
}

Status: 201 Created Size: 195 Bytes Time: 49 ms
{
  "mensaje": "Bug creado",
  "bug": {
    "codigo": 2,
    "proyectoCodigo": 3,
    "descripcion": "Error en el login",
    "severidad": "critica",
    "desarrolladorAsignado": 1002,
    "estado": "abierto",
    "fechaCreacion": "2026-06-10"
  },
  "fechaCreacion": "2026-06-10"
}

```



```

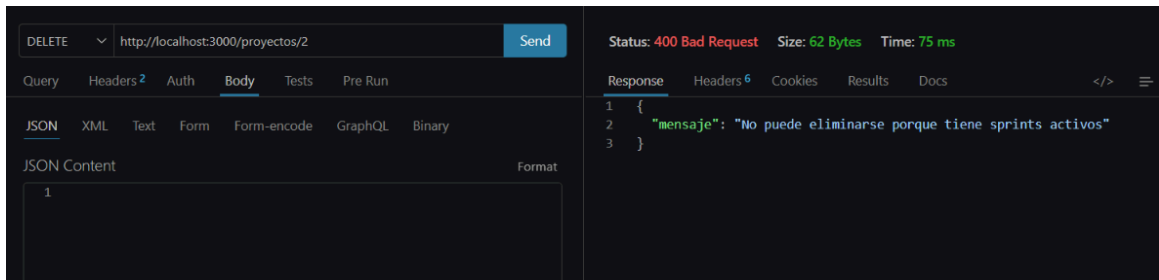
PUT http://localhost:3000/bugs/2
{
  "estado": "resuelto"
}

Status: 200 OK Size: 201 Bytes Time: 141 ms
{
  "mensaje": "Bug actualizado",
  "bug": {
    "codigo": 2,
    "proyectoCodigo": 3,
    "descripcion": "Error en el login",
    "severidad": "critica",
    "desarrolladorAsignado": 1002,
    "estado": "resuelto",
    "fechaCreacion": "2026-06-10"
  },
  "fechaCreacion": "2026-06-10"
}

```

7.2 Prueba Funcional 2: Verificación de Reglas de Negocio

Campo	Detalle
Objetivo	Verificar que las reglas de negocio RF05 y RF06 se aplican correctamente: no eliminar proyectos con sprints activos y no crear sprints para proyectos entregados.
Datos de Entrada	Proyecto 3 con Sprint 2 en estado 'activo' (creado en Prueba 1). Estado destino para cierre: sprint 'completado', proyecto 'entregado'.
Procedimiento	1. DELETE /api/proyectos/3 – Intentar eliminar el proyecto con sprint activo. 2. Verificar respuesta de error. 3. PUT /api/sprints/1 – Cambiar estado del Sprint 1 a 'completado'. 4. PUT /api/proyectos/3 – Cambiar estado del proyecto a 'entregado'. 5. POST /api/sprints – Intentar crear un nuevo sprint para 3.
Resultado Esperado	Paso 1: HTTP 400 con mensaje 'No se puede eliminar el proyecto porque tiene sprints activos'.
Resultado Obtenido	Ambas restricciones funcionaron correctamente. El servidor retornó HTTP 400 con mensajes descriptivos. Los estados en los archivos JSON reflejan los cambios realizados en los pasos intermedios.



8. Resultados Obtenidos

Al término del desarrollo e implementación de NexCode, los resultados obtenidos pueden sintetizarse en los siguientes aspectos:

#	Aspecto Evaluado	Resultado
1	Cobertura de endpoints	Se implementaron 20 endpoints (5 por módulo: GET all, GET by ID, POST, PUT, DELETE) cubriendo el 100% de los requerimientos funcionales.
2	Reglas de negocio	Las 5 reglas de negocio definidas fueron implementadas y validadas exitosamente, retornando los códigos HTTP y mensajes apropiados.
3	Persistencia de datos	Los 4 archivos JSON (proyectos.json, desarrolladores.json, sprints.json, bugs.json) funcionan como almacén de datos con lectura y escritura sincrónicas.
4	Pruebas funcionales	Las 2 pruebas funcionales diseñadas fueron ejecutadas satisfactoriamente, validando flujos positivos y de error.
5	Estructura del proyecto	El código está organizado de forma modular con separación clara entre rutas y datos, facilitando el mantenimiento.

9. Conclusiones

1. La arquitectura REST demostró ser adecuada para la gestión de recursos modulares.

La implementación de NexCode sobre Express.js confirmó que la arquitectura REST facilita la construcción de APIs escalables y comprensibles, donde cada recurso (Proyecto, Desarrollador, Sprint, Bug) tiene una ruta base bien definida y las operaciones sobre él siguen un patrón uniforme de verbos HTTP, lo que reduce la curva de aprendizaje para los equipos que consumen la API.

2. La separación de responsabilidades mediante rutas modulares mejora la mantenibilidad del código.

Al organizar el proyecto en archivos de rutas independientes para cada entidad, se logró un código más limpio y fácil de modificar. Cada módulo puede ser actualizado, reemplazado o extendido de forma independiente sin afectar al resto del sistema, lo que es un principio fundamental en el desarrollo de software profesional.

3. La persistencia en archivos JSON es funcional a nivel académico pero presenta limitaciones en producción.

Si bien el uso de archivos JSON como mecanismo de almacenamiento permitió cumplir los objetivos del proyecto sin depender de infraestructura adicional, esta aproximación presenta desafíos en escenarios de concurrencia, volumen de datos elevado y consistencia transaccional. Para entornos productivos, la migración a un motor de base de datos como MongoDB o PostgreSQL sería imperativa.

4. Las reglas de negocio implementadas refuerzan la integridad referencial del sistema.

La implementación de validaciones lógicas (como verificar la existencia de proyectos antes de crear sprints, o impedir la eliminación de proyectos con sprints activos) demuestra que es posible mantener la coherencia de los datos incluso sin las restricciones de integridad referencial que ofrecen los sistemas de gestión de bases de datos relacionales. Estas validaciones deben diseñarse cuidadosamente en la capa de lógica de negocio.

5. Swagger transforma la documentación de una tarea secundaria en un componente integral del ciclo de desarrollo.

La integración de Swagger desde las primeras etapas del desarrollo permitió disponer de documentación siempre actualizada, reducir los errores de integración por parte de consumidores de la API y facilitar las pruebas sin necesidad de herramientas externas. Esta práctica debe convertirse en un estándar en cualquier proyecto que exponga una API REST.

6. Thunder Client como herramienta de pruebas integrada en el IDE agiliza el ciclo de desarrollo.

La posibilidad de diseñar, ejecutar y documentar colecciones de pruebas HTTP directamente desde Visual Studio Code, sin necesidad de alternar entre aplicaciones, reduce la fricción en el ciclo de prueba-corrección-validación, mejorando la productividad del desarrollador y facilitando la reproducibilidad de las pruebas.

10. Recomendaciones

1. Migrar la persistencia hacia una base de datos NoSQL o relacional.

Se recomienda reemplazar la persistencia en archivos JSON por MongoDB (para mantener la naturaleza documental de los datos) o PostgreSQL (si se requiere integridad relacional fuerte). Esta migración puede realizarse con mínimos cambios en la lógica de rutas si se estructura adecuadamente la capa de acceso a datos desde el inicio.

2. Implementar una capa de controladores separada de las rutas.

Para proyectos de mayor envergadura, se recomienda separar la lógica de negocio de la definición de rutas, creando controladores independientes (archivos controllers/) que gestionen las operaciones sobre los datos. Esto mejora la testabilidad del código y permite reutilizar la misma lógica en múltiples rutas o contextos.

3. Incorporar autenticación y autorización mediante JWT.

La API actual no implementa mecanismos de seguridad. Para un despliegue en entornos no académicos, se recomienda integrar JSON Web Tokens (JWT) para autenticar usuarios y definir roles de acceso que restrinjan las operaciones disponibles según el perfil del consumidor (administrador, desarrollador, cliente).

4. Agregar validación de esquemas con una librería como Joi o Zod.

Actualmente las validaciones de entrada son manuales. Se recomienda incorporar una librería de validación de esquemas como Joi o Zod que permita definir de forma declarativa las reglas de validación para cada entidad, reduciendo el código repetitivo y facilitando la generación de mensajes de error consistentes y descriptivos.

5. Implementar pruebas automatizadas con Jest y Supertest.

Las pruebas manuales con Thunder Client deben complementarse con pruebas automatizadas que puedan ejecutarse en un pipeline de integración continua. Jest, combinado con Supertest para pruebas de integración HTTP sobre Express.js, permite cubrir todos los endpoints y reglas de negocio de forma reproducible y sin intervención manual.

6. Configurar variables de entorno con dotenv.

El puerto del servidor y las rutas de los archivos JSON deben externalizarse mediante variables de entorno usando el paquete dotenv. Esto facilita el despliegue en diferentes entornos (desarrollo, pruebas, producción) sin necesidad de modificar el código fuente.